

The LogicScript Optimizing Compiler

Peien Liang

Hongkong University of Science and
Technology(Guangzhou)
Guangzhou, Guangdong, CHN
pliang812@connect.hkust-gz.edu.cn

Jiakai Li

Hongkong University of Science and
Technology(Guangzhou)
Guangzhou, Guangdong, CHN
jli196@connect.hkust-gz.edu.cn

Zhao Tong

Hongkong University of Science and
Technology(Guangzhou)
Guangzhou, Guangdong, CHN
ztong837@connect.hkust-gz.edu.cn

Abstract

Compilers rely on discrete mathematics for code translation and optimization. This work presents a four-phase optimizing compiler for the custom micro-language LogicScript, implemented in Python. The pipeline validates source code structure, generates a recursive abstract syntax tree, applies propositional logic equivalences to optimize code, and executes instructions via a set/relation-based state environment. Integrating core discrete math concepts (propositional logic, recursive definitions, sets/relations), the compiler verifies optimization correctness through truth table equivalence checking, demonstrating the practical application of theoretical discrete mathematics to real-world compiler design.

1 Introduction

Compilers and interpreters are foundational to modern software, with their code translation and optimization processes deeply rooted in discrete mathematics. This project implements a four-phase optimizing compiler for the custom micro-language LogicScript, built in Python without external parsing or Abstract Syntax Tree (AST) libraries. The modular pipeline validates the lexical and syntactic integrity of LogicScript source code, applies propositional logic equivalences to statically optimize logical expressions, and executes optimized instructions via a set/relation-based state environment. Drawing on core discrete mathematics principles—propositional logic for equivalence verification, recursive definitions for AST generation, and sets/relations for state management—the compiler verifies optimization correctness through truth table checking, handles errors gracefully, and outputs standardized results. This work bridges theoretical discrete mathematics learning with practical software engineering, demonstrating the mathematical foundations of compiler design.

2 System Design

2.1 Overall Structure

The LogicScript compiler comprises four integrated modular components—Lexer, Parser, Optimizer and Executor that collectively process, optimize, and execute source code.

- **Lexer:** performs lexical analysis to tokenize input source code, accurately identifying predefined keywords (let, if, then, print), Boolean operators (AND, OR, NOT, IMPLIES), Boolean literals (T, F), single-lowercase-letter variables (a–z), and structural symbols (=, (,)).
- **Parser:** conducts syntactic analysis on the generated token stream to construct a well-formed Abstract Syntax Tree (AST), supporting core language constructs including variable assignment statements, conditional control flow, print

output operations, and nested logical expressions composed of variables, Boolean values, and parenthesized operations.

- **Optimizer:** statically simplifies logical expressions within the AST by applying fundamental propositional logic equivalence laws, including double negation elimination, De Morgan’s laws, implication conversion, identity, idempotent, absorption, universal bound, and negation laws, alongside constant folding for further reduction.
- **Executor:** validates the correctness of optimization by verifying logical equivalence between original and optimized expressions via exhaustive truth table generation, executes the simplified AST-based program, and maintains real-time tracking of variable state while capturing and storing print output results.

2.2 Key Data Structure

A detailed explanation of the specific data structures used to build the Recursive Parser and the State Dictionary.

- **Recursive Parser:** The parser transforms tokenized input into Abstract Syntax Trees (ASTs) using a prefix-notation, operator-first data structure. This design choice is critical because it preserves the recursive structure of fully parenthesized LogicScript expressions in a form that is easy to traverse and manipulate. For example, a source expression such as $(p \text{ AND } q)$ is converted into $[\text{'AND'}, \text{'VAR_P'}, \text{'VAR_Q'}]$. Once parsed, the AST no longer needs to store parentheses explicitly, because the hierarchical nesting of Python lists already captures grouping unambiguously. This structure directly implements the Recursive Definitions introduced in Chapter 5, and it allows both the optimizer and executor to process the logic tree through clean recursive calls rather than more complicated manual stack handling.
- **State Dictionary:** During the execution phase, the system maintains a state dictionary that functions as a mathematical relation mapping variables to their truth values. More precisely, it can be viewed as a partial function $S : Var \rightarrow Bool$, where Var is the set of program variables and $Bool = \{TRUE, FALSE\}$. It is partial because, at any given point in execution, some variables may not yet have been assigned by a preceding LET statement. In implementation, this function is realized as a Python dictionary, allowing efficient updates and lookups as the program executes line by line. This is essential because the compiler must track which variables are currently in the domain of the state environment and what truth values they hold. Equivalently, the state can be understood as a relation $S \subseteq Var \times Bool$ with the functional property that each variable is associated

with at most one truth value at a time. As a result, the executor can update the environment during LET assignments and retrieve values efficiently during expression evaluation and PRINT operations. This design provides a concrete implementation of the sets-and-relations concepts covered in the course while supporting the sequential execution model required in Phase 4.

2.3 Architectural Innovation: Paradigm Separation

The compiler deliberately combines recursion and iteration at different levels of the pipeline to optimize both structural integrity and execution efficiency. Recursion is employed for expression-level tasks—including parsing, optimization, and evaluation—because LogicScript expressions and AST nodes are recursively defined and naturally form a hierarchical tree structure. In contrast, iteration is utilized for program-level tasks, such as scanning source lines, processing statement sequences, and managing the four distinct compiler phases. These operations follow a linear, line-by-line execution model that is better suited for iterative processing. This strategic separation improves conceptual clarity and ensures that each component of the compiler architecture precisely matches the formal structure of the problem it solves.

3 Testing Suite

To verify the functional correctness and error-handling robustness of the LogicScript compiler, this section presents a structured testing suite, covering testing objectives, valid program compilation verification, lexical and syntactic error detection assessment, and a summary of all test results. Overall, the evaluation covers 52 valid test cases, 6 lexical error cases, 18 syntax error cases, and 3 execution error cases. A complete list and detailed description of these test cases are provided in the Appendix, specifically in test_cases.md.

- **Testing Objectives:** The core objectives of the testing suite are as follows:
 - Validate end-to-end functional correctness of the compiler for well-formed LogicScript programs.
 - Verify robust detection and graceful reporting of lexical and syntactic anomalies without runtime crashes.
 - Ensure strict compliance with the standardized JSON output schema for both successful executions and error-handling scenarios.
- **Valid Input Evaluation:** A representative valid LogicScript test program integrating variable assignments, conditional statements, print operations, and nested logical expressions is utilized to assess full-pipeline integrity. This evaluation confirms the accuracy of lexical tokenization, syntactic Abstract Syntax Tree (AST) generation, logical expression optimization, equivalence verification, and program execution, with the expected outcome being complete pipeline processing and a valid structured output.
- **Lexical Anomaly Testing:** Test cases are designed to induce typical lexical errors, including invalid characters, unrecognized keywords or operators, and non-conformant variable formulations. The compiler is required to terminate

Table 1: LogicScript Compiler Test Case Classification and Output Rules

Category	Input Pattern	Example	Key Output
Successful Compilation	Valid Boolean logic statements: single lowercase variables; uppercase keywords/operators; paired parentheses; complete let/if-then/print structure	let p = T; print p	phase_1_lexer: valid tokens phase_2_parser: valid AST phase_3_optimizer: optimized AST phase_4_execution: output TRUE
Lexical Error Detection	Illegal token-level input: wrong keywords, uppercase variables, multi-character variables, lowercase operators/Booleans, unsupported symbols	assign p = T; let P = T	error in phase_1_lexer no parser/optimizer/execution output
Syntax Error Detection	Lexically valid tokens but grammatically invalid structure: mismatched parentheses, incomplete expressions, missing then, unparenthesized logic	let x = p AND; if p print q	phase_1_lexer: valid tokens error in phase_2_parser no AST / optimized AST / execution output
Execution Error Detection	Semantically valid statements with undefined variables during evaluation	let x = (p AND q); print x	phase_1_lexer: valid tokens phase_2_parser: valid AST phase_3_optimizer: optimized AST error in phase_4_execution, line 1

gracefully at the lexical analysis phase, preserve outputs of successfully completed phases, and generate a precise JSON error entry indicating the lexical phase and error line, with no unhandled runtime exceptions.

- **Syntactic Anomaly Testing:** Syntactic error test cases target structural violations such as mismatched parentheses, missing operands or keywords, and malformed statements. The expected behavior is early termination at the parsing phase, retention of valid preceding-phase outputs, and a clear JSON error record marking the syntactic phase and exact error line, ensuring stable and predictable error handling.

In conclusion, all test cases validate that the compiler delivers reliable end-to-end compilation for valid inputs and robust anomaly identification for defective inputs. The system fully satisfies the project’s functional specifications and error-handling mandates, adhering to the required output format and operational constraints.

4 Limitations

Despite fully complying with the project specifications, the LogicScript compiler exhibits several inherent limitations related to computational scalability, diagnostic precision, optimization scope and future system extensibility.

- **Computational Scalability:** Equivalence verification uses brute-force 2^n truth-table generation with exponential time complexity. This causes significant performance degradation and high memory overhead when handling

expressions with even a moderate number of variables, severely restricting scalability.

- **Diagnostic Precision:** Error detection in the lexer and parser provides only line-level localization without column-level accuracy. This reduces debugging precision and makes it harder to locate and correct malformed code quickly. Further improvement may include column-level accuracy or word-located suggestions.
- **Optimization Scope:** The present optimizer utilizes direct structural pattern matching rather than comprehensive logical normalization procedures. As a result, some expressions that are logically equivalent yet differ in operand arrangement may not be subject to additional simplification.
- **Future System Extensibility:** Core grammatical rules and operator logic are hard-coded, resulting in poor extensibility. Any modification to the language structure or addition of new features requires extensive, invasive code changes.

5 Collaboration

This LogicScript optimizing compiler project was completed by our three-member team, with work structured around the compiler's

modular four-phase pipeline to balance efficient individual development and shared collective ownership. Responsibilities were split to align with core pipeline components: Peien Liang led lexical analysis and error handling, Jiakai Li owned syntax validation and AST generation, and Zhao Tong developed the optimization pass and execution engine, with all members contributing equally to test case design and code integration.

To ensure seamless cross-phase functionality and shared mastery of the full system, the team used Git for version control, held regular sync meetings to resolve technical challenges, and conducted peer code reviews for all modules. Each component lead shared walk-throughs of their work, ensuring every member understood the full compiler architecture and underlying discrete mathematics principles. The team collaborated jointly on drafting the technical report, finalizing the codebase, and preparing the project demonstration, resulting in a cohesive, fully functional implementation.

6 Appendix

The complete test cases and corresponding results are available at <https://github.com/Amireux-bit/Discrete-Mathematics-Project/tree/two-files-only>.